

The Toolbox: Category Rendering, Click-to-Add, and Drag-and-Drop

A Custom HTML Toolbox Over Blockly, and How a Block Enters the Workspace

Research note — Block-MiniJava

2026-07-20

Abstract

Block-MiniJava does not use Blockly’s built-in flyout-based toolbox. The category list, search box, and every draggable block button in `#toolbox-column` are hand-rolled HTML (`src/core/ui/app.ts`), driven by a small block/category registry (`src/core/blocks/minijava Blocks.ts`) and native HTML5 drag-and-drop. A block can enter the workspace two ways — a click, or a drag — and both funnel through the same block-creation function. This note documents the registry, the rendering, both entry paths (including the exact client-to-workspace coordinate conversion drag-and-drop needs), and the “required blocks” mechanism that keeps exactly one `Goal` and one `MainClass` in the workspace at all times, self-healing after any edit, and correspondingly *absent* from the toolbox itself.

Contents

1 Why a custom toolbox	1
2 The block and category registry	2
2.1 The six categories	2
3 Rendering the toolbox	3
4 Entry path 1: click to add	3
5 Entry path 2: drag and drop	4
5.1 Starting the drag: the toolbox button	4
5.2 Receiving the drag: the workspace area	5
5.3 Placing the block: client-to-workspace coordinates	5
6 Grammar-aware connections: what happens after the block lands	6
7 Required blocks: Goal and MainClass	7
8 Full inventory	7
9 HTML wiring	8
Note on scope	8

1 Why a custom toolbox

Blockly ships its own toolbox/flyout system (an XML or JSON toolbox definition, rendered by Blockly itself as a sidebar flyout you drag out of). Block-MiniJava does not use it. Instead,

`#toolbox-column` (`index.html:106--163`) is ordinary HTML that `app.ts` populates and wires by hand: a search input, then one collapsible `<section class="toolbox-category">` per category, each holding a `<button class="toolbox-block-button">` per block. This buys three things a stock Blockly flyout does not give for free in this project’s IDE-shell design: the toolbox can live inside the same activity-bar/sidebar system as the Run and Settings views (`ui-design-and-layout.tex` §2–§3), the search box can filter by block *label* text rather than only by category, and every block button can be a real HTML *draggable* element participating in the browser’s native drag-and-drop rather than Blockly’s own (touch-oriented) flyout drag gesture.

2 The block and category registry

`src/core/blocks/minijavaBlocks.ts` is the single source of truth for which block types exist and how the toolbox groups them — the `renderToolbox` function (§3) never hard-codes a block type or category; it only iterates this data.

Symbol	Role
<code>ToolboxBlock</code>	<code>minijavaBlocks.ts:4--8</code> — <code>{type, label, icon}</code> : a block type name, its toolbox display label, and a short glyph/text icon.
<code>ToolboxCategory</code>	<code>minijavaBlocks.ts:10--15</code> — <code>{id, label, icon, blocks: ToolboxBlock[]}</code> .
<code>MINI_JAVA_REQUIRED_BLOCK_TYPES</code>	<code>minijavaBlocks.ts:17</code> — the two-element tuple <code>['mj_goal', 'mj_main_class']</code> , destructured in <code>app.ts:62</code> into <code>GOAL_BLOCK_TYPE/MAIN_BLOCK_TYPE</code> .
<code>MINI_JAVA_BLOCK_TYPES</code>	<code>minijavaBlocks.ts:19--59</code> — the full list of 38 block type names (grammar non-terminals, statements, expressions, literals); the type <code>MiniJavaBlockType</code> (l.61) is its element union.
<code>MINI_JAVA_CATEGORIES</code>	<code>minijavaBlocks.ts:67--144</code> — six <code>ToolboxCategory</code> entries, in display order: Program , Declarations , Types , Statements , Expressions , Values (§2.1).
<code>defineMiniJavaBlocks()</code>	<code>minijavaBlocks.ts:155--(end)</code> — registers every block’s JSON definition with Blockly (<code>Blockly.common.defineBlocksWithJsonArray</code>), including each input’s grammar-aware connection <i>check</i> (§6) — called once from <code>app.ts</code> before the workspace is injected.

2.1 The six categories

Category	id	Blocks
Program	<code>program</code>	<code>mj_class_declaration</code> .
Declarations	<code>declarations</code>	<code>mj_var_declaration</code> , <code>mj_method_declaration</code> , <code>mj_formal_parameter</code> .
Types	<code>types</code>	<code>mj_type_int_array</code> , <code>mj_type_boolean</code> , <code>mj_type_int</code> , <code>mj_type_string</code> , <code>mj_type_identifier</code> .

Category	id	Blocks
Statements	statements	<code>mj_statement_block</code> , <code>mj_statement_if</code> , <code>mj_statement_while</code> , <code>mj_statement_print</code> , <code>mj_statement_assign</code> , <code>mj_statement_array_assign</code> .
Expressions	expressions	<code>mj_expr_arith</code> , <code>mj_expr_compare</code> , <code>mj_expr_logic</code> , <code>mj_expr_array_lookup</code> , <code>mj_expr_array_length</code> , <code>mj_expr_char_at</code> , <code>mj_expr_concat</code> , <code>mj_expr_str_length</code> , <code>mj_expr_method_call</code> , <code>mj_argument_item</code> , <code>mj_expr_not</code> , <code>mj_expr_parens</code> .
Values	values	<code>mj_expr_integer</code> , <code>mj_expr_string</code> , <code>mj_expr_boolean</code> , <code>mj_expr_identifier</code> , <code>mj_expr_this</code> , <code>mj_expr_new_int_array</code> , <code>mj_expr_new_object</code> .

Two block types from `MINI_JAVA_BLOCK_TYPES` are conspicuously absent from every category: `mj_goal` and `mj_main_class` — see §7 for why. `mj_value_object`, `mj_value_null`, and `mj_viz_description` are also absent: the first two are runtime-only value blocks the substitution stepper renders into its own private workspace (`typing-and-evaluation-rules.tex` §4.3, `bottom-panel-sync.tex` §2.4), never user-authored; the third is a visualization annotation block, also never user-placed directly.

3 Rendering the toolbox

Function	Role
<code>renderToolbox(query)</code>	<code>app.ts:830--898</code> — rebuilds <code>#toolbox-content</code> from scratch on every call (no incremental diffing — the whole list is cheap enough to throw away and rebuild). With no query, every category and block renders. With a query, a category whose own label/id matches shows <i>all</i> its blocks; otherwise each block is individually filtered by whether its label or type contains the (lower-cased) query (l.837–843) — so searching “if” shows the <code>if</code> / <code>else</code> statement even though its category label (“Statements”) does not itself match. A category that ends up with zero visible blocks is omitted entirely (l.844); if <i>no</i> block anywhere matches, a <code>.toolbox-search-empty</code> message is shown instead (l.891–897).
<code>addBlockToWorkspace(type)</code>	<code>app.ts:900--902</code> — the click handler installed on every toolbox button (l.872); see §4.

Each category is a collapsible `<section>`: its `<button class="toolbox-category-header">` toggles a `collapsed` class on click (`app.ts:855--858`), independent per category, with no persisted collapse state — collapsing is a same-session convenience, not a saved preference.

4 Entry path 1: click to add

Function	Role
<code>addBlockToWorkspace(type)</code>	<code>app.ts:900--902</code> — <code>createBlockInWorkspace(type, activeWorkspaceInsertionPoint())</code> .
<code>activeWorkspaceInsertionPoint()</code>	<code>app.ts:178--185</code> — picks a point inset from the visible workspace viewport's top-left corner (18% of the viewport's width/height, clamped to [72, 140]px horizontally and [72, 120]px vertically, l.180–181), then adds a small cascading offset that increments by 24px (mod 120) on every call (<code>clickAddOffset</code> , l.50, l.182–183) — so repeatedly clicking the same toolbox button drops a visible <i>staircase</i> of blocks rather than stacking them exactly on top of each other.
<code>createBlockInWorkspace(type, position)</code>	<code>app.ts:187--205</code> — if <code>type</code> is the Goal or MainClass type, redirects to <code>ensureRequiredBlocks</code> instead of creating a duplicate (a defensive branch: neither type is ever reachable through the toolbox UI at all, §7, so this only guards a hypothetical direct call). Otherwise: opens a <code>Blockly.Events</code> group (l.193, so the create + move the caller performs together undo as one user action), calls <code>workspace.newBlock(type)</code> , <code>initSvg()</code> , <code>render()</code> , <code>moveBy(position.x, position.y)</code> , and <code>select()</code> , then calls <code>updateCode()</code> and <code>saveAutosave()</code> .

`clientPointToWorkspacePoint` (§5.3) is *not* used on this path — a plain click has no client coordinate to convert, only "somewhere sensible near the top-left of what's currently visible."

5 Entry path 2: drag and drop

Every toolbox block button is a native HTML5 draggable element (`button.draggable = true`, `app.ts:869`); dropping it onto the Blockly canvas creates a block at the exact drop point. This is ordinary browser drag-and-drop (the `DataTransfer` API), not any Blockly-provided drag gesture.

5.1 Starting the drag: the toolbox button

Event/Code	Role
<code>TOOLBOX_BLOCK_MIME</code>	<code>app.ts:61</code> — the custom MIME type " <code>application/x-block-minijava-block</code> " used to carry the block type string through the drag.
<code>dragstart</code>	<code>app.ts:873--878</code> — adds the <code>is-dragging</code> CSS class for visual feedback; calls <code>event.dataTransfer.setData</code> twice — once under <code>TOOLBOX_BLOCK_MIME</code> (the precise channel) and once under the standard <code>'text/plain'</code> (a fallback, in case something in the drop path only reads the generic type); sets <code>effectAllowed = 'copy'</code> (dragging a toolbox block never removes it from the toolbox — it is always a copy/instantiate operation, never a move).

Event/Code	Role
<code>dragend</code>	<code>app.ts:879--882</code> — removes <code>is-dragging</code> and clears the <code>workspace-drop-target</code> highlight regardless of whether the drop actually succeeded (a drag cancelled outside any valid target still fires <code>dragend</code>).

5.2 Receiving the drag: the workspace area

Function	Role
<code>hasToolboxBlockDrag(event)</code>	<code>app.ts:908--910</code> — true iff the in-flight drag's <code>dataTransfer.types</code> includes <code>TOOLBOX_BLOCK_MIME</code> — the guard every handler below uses first, so dragging some unrelated thing (a browser tab, a file, page text) over the canvas is silently ignored rather than mishandled.
<code>initToolboxDragAndDrop()</code>	<code>app.ts:912--941</code> — installs four listeners on <code>#blockly-area</code>: <code>dragenter</code> (1.915–919) — <code>preventDefault()</code> (required for the element to become a valid drop target at all under the HTML5 DnD spec) and adds <code>workspace-drop-target</code> for a visible drop-zone highlight. <code>dragover</code> (1.921–926) — also <code>preventDefault()</code> (fired continuously while hovering; the spec requires this every time, not just on <code>dragenter</code>), and sets <code>dropEffect = 'copy'</code> so the OS cursor shows a copy (not move) affordance. <code>dragleave</code> (1.928–931) — removes the highlight only when the pointer has left <code>#blockly-area</code> entirely (checked via <code>event.relatedTarget</code>, so moving between child elements <i>inside</i> the drop zone does not flicker the highlight off and back on). <code>drop</code> (1.933–940) — <code>preventDefault()</code> (stops the browser's default handling, e.g. navigating to dropped text as a URL), clears the highlight, reads the block type back out of <code>dataTransfer</code> (custom MIME first, <code>'text/plain'</code> as fallback), and calls <code>addBlockToWorkspaceAtClientPoint(type, event.clientX, event.clientY)</code>.

5.3 Placing the block: client-to-workspace coordinates

A drop event's `clientX/clientY` are **screen-space pixel** coordinates; a Blockly block needs a position in **workspace-space** coordinates (which account for the canvas's current pan offset and zoom level). This conversion is the one genuinely non-trivial piece of the whole toolbox implementation.

Function	Role
<code>blocklyViewportRect()</code>	<code>app.ts:159--161</code> — <code>#blockly-div</code> ’s <code>getBoundingClientRect()</code> : the on-screen rectangle the canvas currently occupies.
<code>clientPointToWorkspacePoint(clientX, clientY)</code>	<code>app.ts:163--176</code> — the conversion, in four steps: (1) read <code>workspace.getMetrics()</code> for the current <code>viewLeft/viewTop</code> (how far the canvas is scrolled/panned) and <code>currentScale()</code> for the zoom factor; (2) clamp the client point into the viewport rectangle first (<code>Math.max(0, Math.min(...))</code> , 1.170–171) so a drop event that lands slightly outside the canvas element (a few pixels of border/padding) still resolves to a point <i>inside</i> it, rather than a nonsensical negative or out-of-range workspace coordinate; (3)-(4) convert the clamped local pixel offset to workspace units: $x_{ws} = \text{viewLeft} + \text{localX}/\text{scale}$, and symmetrically for y (1.172–175) — dividing by scale is what makes a drop land on the <i>same visual spot</i> regardless of whether the canvas is zoomed in or out.
<code>addBlockToWorkspaceAtClientPoint(type, clientX, clientY)</code>	<code>app.ts:904--906</code> — <code>createBlockInWorkspace(type, clientPointToWorkspacePoint(clientX, clientY))</code> ; the drag-and-drop counterpart of <code>addBlockToWorkspace</code> (§4), sharing the same <code>createBlockInWorkspace</code> block-creation logic — the only difference between the click path and the drag path is <i>which</i> position function computes where the new block lands.

`currentScale()` (`app.ts:150--156`) reads Blockly’s own `getScale()` when available, falling back to a raw `.scale` property read for older/differently-typed Blockly builds — defensive compatibility code, not part of the coordinate math itself.

6 Grammar-aware connections: what happens after the block lands

Neither the click nor the drag path checks *where* the new block is allowed to connect — a freshly created block always lands as an unconnected top-level block at the computed position. Whether it can subsequently be dragged *into* a connection with an existing block is governed entirely by Blockly’s own connection-compatibility check against the `check` metadata every block input declares in `defineMiniJavaBlocks()` (`minijavaBlocks.ts`, 40 `check`: sites plus typed `output/previousStatement/nextStatement` declarations) — named after MiniJava grammar non-terminals (`MainClass`, `ClassDeclaration`, `VarDeclaration`, `MethodDeclaration`, `FormalParameter`, `Type`, `Statement`, `Expression`, `Identifier`), exactly as the README’s Grammar section and the project’s custom BMJ-Thrasos renderer (`core/renderer/`) describe. This note does not re-derive that system — it is the same static, structural check documented in the README’s “Grammar-aware renderer” section — but it is worth stating precisely how the toolbox interacts with it: **the toolbox creates blocks; the connection-check system, entirely separate and already built into Blockly plus this project’s check metadata, decides where they may snap.** No toolbox code inspects or enforces grammar.

7 Required blocks: Goal and MainClass

Every MiniJava program has exactly one **Goal** (the program root) wrapping exactly one **MainClass**. Rather than ask the user to place these two blocks correctly (and keep them that way), Block-MiniJava enforces their existence and wiring automatically, and — as already noted in §2.1 — never offers them in the toolbox at all, so there is no user action that could create a *second* Goal or a disconnected MainClass in the first place; the enforcement machinery below is a safety net against indirect ways a workspace could still end up without them (loading an old/hand-edited save file, an incomplete text-editor import, an aborted paste).

Function	Role
<code>lockRequiredBlock(block)</code>	<code>app.ts:255--259</code> — <code>setDeletable(false)</code> , <code>setMovable(false)</code> , <code>setCollapsed(false)</code> : once a Goal or MainClass block exists, the user can edit its contents (attach classes, fill in the class name) but can neither delete it nor drag it away from its required position.
<code>createRequiredBlock(ws,type,x,y)</code>	<code>app.ts:261--267</code> — the same <code>newBlock/initSvg/render/moveBy</code> sequence as <code>createBlockInWorkspace</code> (§4), factored out since required blocks are created from a different call site with no click/drag position to reuse.
<code>removeDuplicateRequiredBlock(block)</code>	<code>app.ts:269--272</code> — <code>setDeletable(true)</code> then <code>dispose(false)</code> : a <i>second</i> Goal or MainClass block (e.g. from a merge of two saved files) is un-locked just long enough to be disposed of, keeping exactly one of each.
<code>ensureRequiredBlocks(ws)</code>	<code>app.ts:274--327</code> — the sweep, guarded against re-entrancy (<code>enforcingRequiredBlocks</code> , l.51, l.275–276, since disposing a block or moving a connection inside this function itself fires more workspace-change events that would otherwise re-trigger it): finds (or creates) the one Goal; disposes every extra Goal; finds (or creates) the MainClass <i>connected to that Goal</i> in preference to any other stray MainClass on the workspace (l.297–298); disposes every other MainClass; if the Goal and MainClass are not already connected to each other, forcibly disconnects whatever <i>is</i> on each connection first and connects them (l.312–319); finally locks both via <code>lockRequiredBlock</code> . Returns whether anything actually changed.
<code>scheduleRequiredBlockEnforcement()</code>	<code>app.ts:329--336</code> — debounced (a zero-delay <code>setTimeout</code> , so it runs once after the current synchronous event-handling batch finishes rather than immediately inside a Blockly event callback) call to <code>ensureRequiredBlocks</code> ; calls <code>updateCode()</code> only if something changed. Invoked from the workspace’s change listener on every mutation event (<code>app.ts:1112,1121</code>) and from <code>FINISHED_LOADING</code> (a full workspace load, e.g. Open or autosave restore, l.1111–1113).

8 Full inventory

Path	Position function	Shared creation call
Click a toolbox button	<code>activeWorkspaceInsertionPoint()</code>	<code>createBlockInWorkspace(type, position)</code>
Drag a toolbox button onto the canvas	<code>clientPointToWorkspacePoint(event.clientX, event.clientY)</code>	

9 HTML wiring

Element	Location
<code>#toolbox-column</code>	<code>index.html:106--163</code> — the primary sidebar; the <code>blocks/search</code> view (<code>index.html:114--120</code>) holds <code>#toolbox-search</code> and <code>#toolbox-content</code> .
<code>#toolbox-search</code>	<code>index.html:116</code> — wired at <code>app.ts:1219</code> to call <code>renderToolbox(input.value)</code> on every <code>input</code> event.
<code>#toolbox-content</code>	<code>index.html:119</code> — the render target of <code>renderToolbox</code> (§3); empty until first populated at IDE init.
<code>#blockly-area</code>	<code>index.html:200--203</code> — the drop target for <code>initToolboxDragAndDrop</code> (§5); wraps <code>#blockly-div</code> , the actual Blockly-injected SVG canvas.

Note on scope

This note covers block *creation* — the toolbox UI, the click and drag entry paths, and the required-block self-healing invariant. It does not re-derive the grammar-aware connection-check system that governs where a created block may subsequently connect (already documented in the project’s own README, “Grammar-aware renderer” section) or the static type system that separately checks the resulting program (`typing-and-evaluation-rules.tex`). The workbench shell the toolbox sits inside (activity bar, sidebar resizing, responsive collapse) is covered in `ui-design-and-layout.tex`.